CRIPTOGRAFÍA PRIMERO $2026 = 2 * 1013$

EL ALUMNO REALIZARÁ UN PROGRAMA EN C que realice la tarea que se haya encomendado. El **profesor responderá** a cualquier duda o consulta que tenga el alumno.
Pongan su **nombre** y **apellidos** como comentario al principio del programa.

La mayoría de los programas **mal hechos** lo son por programar algo distinto a lo que aquí se ha encomendado. En caso de duda, **pregunten al profesor**.

Fecha límite de entrega: lunes 23 de noviembre. No hay garantías de que justo días previos a esa fecha les pueda atender el profesor. Cada día de retraso penaliza 11 %. Solo se admite la entrega definitiva del programa. No se admiten entregas previas o segundas entregas.
Se valorará **cero** los programas que no funcionen o no proporcionen lo pedido en bastantes casos y los programas **poco legibles por humanos**.
Se valorará escasamente programas que fallen en ciertos casos. Se penalizará el exceso de cuentas, cálculos innecesarios, repetidos,...exceso de variables innecesarias, ineficiencias,...programas poco legibles, sin comentarios explicativos, con estructuración obscura, **malas prácticas de programación**,...
Se puede pedir al alumno que **explique** su funcionamiento y que lo ejecute en presencia del profesor.
Si resultase claro que el alumno no puede explicar el funcionamiento de parte del programa que él ha entregado, puede valorarse **cero**; aún cuando dicho programa esté perfecto [Espero que sepan el porqué].

Solo se pueden usar las librerías usuales y comunes del lenguaje C. En caso de duda preguntar.

Cuidado: El hecho de que tu programa cifre un fichero en otro; y que el programa de descifrado devuelva el fichero original **no es** garantía de que lo hayas realizado bien.

Los programas pueden usar como base el programa en C comentado en clase convenientemente modificado. Deben ser un cifrador y descifrador del estilo al programa comentado. En casi todos los casos el programa de cifrado y de descifrado son algo distintos; el programa debe preguntar en primera instancia **si queremos cifrar o descifrar**. En cada caso se solicita un programa en C que, además [en vez de] de lo que tiene el programa impartido, realice la tarea que se le ha encomendado de entre las siguientes.

---

1 Debe usar un vector de inicialización IV. Dicho vector lo debe pedir el programa: debe consistir en 10 bytes. Al cifrar el fichero cifrado debe tener la siguiente estructura

z[10 bytes]z[el fichero cifrado propiamente]

De esta forma, el tamaño del fichero cifrado crece en 12 bytes sobre el original. Se ponen dos caracteres "z" como control.
El vector de inicialización se puede mezclar con la clave de varias formas: lo más razonable es que se haga un XOR [suma] binaria con los 10 primeros bytes de la clave.
Ahora los programas de cifrado y de descifrado son ligeramente distintos.

2 Debe usar un vector de inicialización IV. Dicho vector lo debe pedir el programa: debe consistir en 2 bytes. Al cifrar el fichero cifrado debe tener la siguiente estructura

a[2 bytes]a[el fichero cifrado propiamente]

De esta forma, el tamaño del fichero cifrado crece en 4 bytes sobre el original. Se ponen dos caracteres "a" como control.
En vez de usar estas sentencias

```
k=registro[3];
 for(j=0; j<k; j++)
  avance();
```

Antes de empezar a cifrar, el programa debe hacer tantos avances del registro como el valor de un número entero $0 \leq k < 2^{16}$, representado por los dos bytes del vector de inicialización.
Ahora los programas de cifrado y de descifrado son ligeramente distintos.

3 Uso del propio mensaje como clave parcial. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor. Son sucesiones de bytes. Pensado cada byte como 8 bits:
Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \cong (\mathbb{F}_2)^8$ componente a componente. También se denomina suma bitwise. Ahora la operación de cifrado será

$$\begin{cases} e_n = m_n \oplus c_n & \text{si} \quad n \leq 8. \\ e_n = m_n \oplus m_{n-8} \oplus c_n & \text{si} \quad n > 8. \end{cases}$$

Nótese que la operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte.

4 Cambio de sucesión cifrante y longitud del registro:
La sucesión cifrante se construye a partir de la sucesión de recurrencia lineal

$$a_{n+17} = a_{n+11} + 16a_n, \qquad \text{operando en} \quad \mathbb{Z}/(599).$$

De forma equivalente

$$a_n = a_{n-6} + 16a_{n-17}, \qquad \text{operando en} \quad \mathbb{Z}/(599).$$

Posteriormente cada valor a_n, que está en el rango $\{0, 1, 2, \ldots, 597, 598\}$, se toma módulo 256 haciendo

$$b_n \equiv a_n \pmod{256}.$$

De forma que cada b_n está en el rango $\{0, 1, 2, \ldots, 254, 255\}$. Y ya con la sucesión (b_n) se opera como en el programa modelo. Ahora la clave debe ser de 17 bytes. Y el registro que almacena la sucesión (a_n) debe tener el correspondiente tamaño.

5 Cambio de la sucesión cifrante y uso de un vector de inicialización. En vez de la sucesión propuesta en programa modelo se usará la sucesión cifrante

$$s_{n+17} = s_{n+4} + 21 * s_n \qquad \text{operando en} \quad \mathbb{Z}/(251).$$

De forma equivalente

$$s_n = s_{n-13} + 21 * s_{n-17} \qquad \text{operando en} \quad \mathbb{Z}/(251).$$

Ahora el registro que soporta la sucesión deberá ser de longitud 17. Se usará una clave de 13 bytes como en el programa modelo. Además se usará un vector de inicialización IV consistente en 4 bytes. El programa debe pedir dicho vector al usuario. Al cifrar el fichero cifrado debe tener la siguiente estructura

w[4 bytes]w[el fichero cifrado propiamente]

De esta forma, el tamaño del fichero cifrado crece en 6 bytes sobre el original. Se ponen dos caracteres "w" como control.
La clave de 13 bytes y el vector de inicialización de 4 bytes, se toman módulo 251 como en el programa modelo, y son los primeros 17 valores del registro que guarda la sucesión. Pongamos por orden primero los 13 de la clave, y luego 4 del IV.
Ahora los programas de cifrado y de descifrado son ligeramente distintos.

6 Uso del propio mensaje como clave parcial. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor. Son sucesiones de bytes. Pensado cada byte como 8 bits:
Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \cong (\mathbb{F}_2)^8$ componente a componente. También se denomina suma bitwise. Y denotamos mediante $\boxplus$ la suma en $\mathbb{Z}/(256)$. Ahora la operación de cifrado será

$$\begin{cases} e_n = m_n \oplus c_n & \text{si} \quad n \leq 5. \\ e_n = [m_n \boxplus m_{n-5}] \oplus c_n & \text{si} \quad n > 5. \end{cases}$$

Nótese que en la última linea hay dos operaciones de **"suma"** distintas. La operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte.

7 Uso del producto(modificado) de dos sucesiones de recurrencia lineal para la sucesión cifrante. Además de la sucesión del programa modelo se usará otra sucesión de recurrencia lineal sobre el cuerpo $\mathbb{Z}/(5)$. Se hará un producto modificado para conseguir la sucesión cifrante. Sea la sucesión de recurrencia lineal

$$b_{n+13} = b_{n+7} + 2 * b_n, \qquad \text{operando en} \quad \mathbb{Z}/(5).$$

De forma equivalente

$$b_n = b_{n-7} + 2 * b_{n-13}, \qquad \text{operando en} \quad \mathbb{Z}/(5).$$

Esta sucesión se debe inicializar con parte de la clave; por ejemplo, tomando cada byte de la clave módulo 5. Si (a_n) es la sucesión en $\mathbb{Z}/(251)$ del programa modelo, nosotros conseguimos la sucesión cifrante multiplicando (con una modificación) ambas módulo 251. La sucesión (b_n) está en el rango $\{0, 1, 2, 3, 4\}$. Para evitar que en la sucesión cifrante (c_n) uno de cada cinco valores valga cero, modificamos el producto así:

$$\begin{cases} c_n = a_n.b_n & \text{si} \quad b_n \in \{1, 2, 3, 4\}. \quad \text{Producto realizado en} \quad \mathbb{Z}/(251). \\ c_n = 5.a_n & \text{si} \quad b_n = 0. \quad \text{Producto realizado en} \quad \mathbb{Z}/(251). \end{cases}$$

Espero que se entienda que sería una debilidad que la sucesión cifrante tuviese una cantidad desproporcionada de ceros.
Ahora la sucesión (c_n) se usa como sucesión cifrante en la misma manera que en el programa modelo.

8 Uso de una suma de dos sucesiones de recurrencia lineal para la sucesión cifrante. Además de la sucesión del programa modelo se usará otra sucesión de recurrencia lineal sobre el mismo cuerpo $\mathbb{Z}/(251)$. Se sumarán de forma peculiar ambas sucesiones para conseguir la sucesión cifrante. Sea la sucesión de recurrencia lineal

$$b_{n+7} = b_{n+2} + 6 * b_n, \qquad \text{operando en} \quad \mathbb{Z}/(251).$$

De forma equivalente

$$b_n = b_{n-5} + 6 * b_{n-7}, \qquad \text{operando en} \quad \mathbb{Z}/(251).$$

Esta sucesión se debe inicializar con parte de la clave.
El registro que almacena la sucesión (b_n) puede tener una longitud de 7 bytes.
Si (a_n) es la sucesión en $\mathbb{Z}/(251)$ del programa modelo, nosotros conseguimos la sucesión cifrante sumando ambas, pero introducimos el truco de sumarlas módulo 256. Entendamos: tanto a_n como b_n están representados en rango $\{0, 1, 2, 3, \ldots, 249, 250\}$ conseguimos la sucesión cifrante (c_n) de la forma

$$c_n \equiv a_n + b_n \pmod{256}.$$

Ahora cada c_n está en el rango $\{0, 1, 2, 3, \ldots, 254, 255\}$. Ahora la sucesión (c_n) se usa como sucesión cifrante en la misma manera que en el programa modelo.

9 Uso de **dos** sucesiones de recurrencia lineal para **dos** sucesiones cifrantes. Una multiplicará y la ya conocida (a_n) hará suma XOR. Además de la sucesión del programa modelo se usará otra sucesión de recurrencia lineal sobre el cuerpo $\mathbb{Z}/(5)$. Se hará un producto modificado para conseguir la sucesión cifrante. Sea la sucesión de recurrencia lineal

$$b_{n+13} = b_{n+7} + 2 * b_n, \qquad \text{operando en} \quad \mathbb{Z}/(5).$$

De forma equivalente

$$b_n = b_{n-7} + 2 * b_{n-13}, \qquad \text{operando en} \quad \mathbb{Z}/(5).$$

Esta sucesión se debe inicializar con parte de la clave; por ejemplo, tomando cada byte de la clave módulo 5. Esta sucesión se "almacena" en otro registro.
Sea (m_n) la sucesión que representa el mensaje claro que debe ser cifrado. La sucesión (b_n) está en el rango $\{0, 1, 2, 3, 4\}$. Para evitar multiplicar por un divisor de cero, consideramos $(2 * b_n + 1)$, que está en el rango $\{1, 3, 5, 7, 9\}$ para multiplicar luego módulo 256 por cada byte del mensaje. El primer paso del cifrado consiste en

$$h_n = m_n.(2b_n + 1) \qquad \text{Producto realizado en} \quad \mathbb{Z}/(256).$$

Si (a_n) es la sucesión en $\mathbb{Z}/(251)$ del programa modelo, la consideramos como una sucesión de bytes, aunque solo toma los valores $\{0, 1, 2, \ldots, 249, 250\}$, el segundo paso del cifrado consiste en

$$e_n = h_n \,\text{XOR}\, a_n, \qquad \text{byte a byte},$$

obteniendo el mensaje cifrado (e_n).

El descifrado se realiza también en dos pasos. Para el descifrado; hay que multiplicar por los inversos de $1, 3, 5, 7, 9$ en $\mathbb{Z}/(256)$, los cuales son por orden $1, 171, 205, 183, 57$, y esto según b_n valga $0, 1, 2, 3, 4$ respectivamente.

10 Uso de cierto producto de dos sucesiones de recurrencia lineal para la sucesión cifrante. Además de la sucesión del programa modelo se usará otra sucesión de recurrencia lineal sobre el cuerpo $\mathbb{Z}/(7)$. Se hará un producto modificado para conseguir la sucesión cifrante. Sea la sucesión de recurrencia lineal

$$b_{n+13} = b_{n+5} + 3 * b_n, \qquad \text{operando en} \quad \mathbb{Z}/(7).$$

De forma equivalente

$$b_n = b_{n-8} + 3 * b_{n-13}, \qquad \text{operando en} \quad \mathbb{Z}/(7).$$

Esta sucesión se debe inicializar con la clave; por ejemplo, tomando cada byte de la clave módulo 7.
Si (a_n) es la sucesión en $\mathbb{Z}/(251)$ del programa modelo, nosotros conseguimos la sucesión cifrante multiplicando (con una modificación) ambas módulo 251. La sucesión (b_n) está en el rango $\{0, 1, 2, 3, 4, 5, 6\}$. Para evitar que en la sucesión cifrante (c_n) uno de cada siete valores valga cero, modificamos el producto así, consideramos $(b_n + 1)$ en el rango $\{1, 2, 3, 4, 5, 6, 7\}$ para multiplicar luego módulo 251:

$$c_n = a_n.(b_n + 1) \qquad \text{Producto realizado en} \quad \mathbb{Z}/(251).$$

Sería una debilidad que la sucesión cifrante tuviese una cantidad desproporcionada de ceros.
Ahora la sucesión (c_n) se usa como sucesión cifrante en la misma manera que en el programa modelo.

11 Shriking generator. Generador menguante. En vez de usar toda sucesión cifrante del programa modelo; se usarán dos tercios de los valores que proporciona. Para ello se utiliza otra sucesión de recurrencia lineal sobre $\mathbb{F}_3 = \mathbb{Z}/(3)$. Sea la sucesión de recurrencia lineal

$$b_{n+13} = b_{n+7} + 2 * b_n, \qquad \text{operando en} \quad \mathbb{Z}/(3).$$

De forma equivalente

$$b_n = b_{n-6} + 2 * b_{n-13}, \qquad \text{operando en} \quad \mathbb{Z}/(3).$$

Esta sucesión se debe inicializar con parte de la clave; por ejemplo, tomando cada byte de la clave módulo 3.
Sea (a_n) la sucesión cifrante proporcionada por el programa modelo. La sucesión cifrante en este caso, (c_n), se consigue a partir de (a_n) y de (b_n) de la siguiente manera:

Suponemos que ya hemos conseguido $\{c_n : 1 \leq n < m\}$. Y (a_n) y (b_n) han avanzado hasta $n = h - 1$. Nótese que ya se ha podido producir una mengua $m \leq h$.
Se calcula el siguiente valor de (c_n). Ahora operamos como sigue:

- Si $b_h = 0$, hacemos un avance en la sucesión (a_n), actualizamos el registro correspondiente, pero el valor a_h no se usa para nada.
- Si $b_h \in \{\bar{1}, \bar{2}\}$, hacemos un avance en la sucesión (a_n), actualizamos el registro correspondiente, y $c_m = a_h$.

El registro que guarda la sucesión (b_n) se debe actualizar en cada iteración.
Ahora la sucesión (c_n) es la que se usa para el cifrado y descifrado de la misma forma que en el programa modelo.
Ahora ya no son interesantes estas instrucciones:

```
k=registro[3];
 for(j=0; j<k; j++)
  avance();
```

12 Uso del propio mensaje para cifrar. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor. Son sucesiones de bytes. Pensado cada byte como 8 bits: Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$ componente a componente. También se denomina suma bitwise.

Nótese que m_n está en el rango $\{0, 1, 2, \ldots, 255\}$ y que c_n está en el rango $\{0, 1, 2, \ldots, 250\}$. Consideramos ambos números como elementos de $\mathbb{Z}/(256)$, es decir, como si ambos estuviesen en el rango $\{0, 1, 2, \ldots, 255\}$. Denotamos mediante $m_{n-1}.c_n$ el resultado del producto en el anillo $\mathbb{Z}/(256)$.

Ahora la operación de cifrado será

$$\begin{cases} e_n = m_n \oplus c_n & \text{para} \quad n = 1. \\ e_n = m_n \oplus [m_{n-1}.c_n] & \text{para} \quad n > 1. \end{cases}$$

Nótese que la operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte.

13 Uso del propio mensaje para cifrar. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor. Son sucesiones de bytes. Pensado cada byte como 8 bits: Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$ componente a componente. También se denomina suma bitwise.

Nótese que m_n está en el rango $\{0, 1, 2, \ldots, 255\}$ y que c_n está en el rango $\{0, 1, 2, \ldots, 250\}$. Consideramos ambos números como elementos de $\mathbb{Z}/(251)$, reduciendo m_n módulo 251.
Llamo al resultado $m'_n \equiv m_n (\text{mód } 251)$. Tanto c_n como m'_n están en el rango $\{0, 1, 2, \ldots, 250\}$.
Denotamos mediante $m'_{n-1} * c_n$ el resultado del producto en el anillo $\mathbb{Z}/(251)$. Y el valor resultante lo consideramos en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$.

Ahora la operación de cifrado será

$$\begin{cases} e_n = m_n \oplus c_n & \text{para} \quad n = 1. \\ e_n = m_n \oplus \left[m'_{n-1} * c_n\right] & \text{para} \quad n > 1. \end{cases}$$

Nótese que la operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte.

14 Uso de dos operaciones para cifrar. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor.

Creamos otra nueva sucesión como se indica:
Sea $g_n \equiv c_n (\text{mód } 10)$, Se tiene que g_n está en el rango $\{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$. Consideramos $(2 * g_n + 1)$, que está en el rango $\{1, 3, 5, 7, 9, 11, 13, 15, 17, 19\}$ para multiplicar luego módulo 256. La primera parte del cifrado consiste en el producto

$$r_n = m_n.(2 * g_n + 1) \qquad \text{Producto realizado en} \quad \mathbb{Z}/(256). \tag{1}$$

La segunda parte del cifrado es como se muestra en el programa modelo proporcionado por el profesor. Para sucesiones de bytes, pensado cada byte como 8 bits: Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$ componente a componente. También se denomina suma bitwise. Realizamos

$$e_n = r_n \oplus c_n \qquad \text{suma bitwise.} \tag{2}$$

El mensaje cifrado es (e_n), que es el que debe escribirse en el fichero cifrado.

Nótese que la operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte. El descifrado debe tener dos fases. La primera es realizar lo indicado en (2), pero intercambiando los papeles de e_n y de r_n.

Lo segundo es multiplicar por los inversos de $\{1, 3, 5, 7, 9, 11, 13, 15, 17, 19\}$ en $\mathbb{Z}/(256)$, que son respectivamente $\{1, 171, 205, 183, 57, 163, 197, 239, 241, 27\}$

15 Cifrado de una parte de los caracteres:
Para evitar el conflicto subjetivo de ver caracteres raros. El nuevo programa solo reemplazará un caracter si su número cumple $33 \leq \text{char} \leq 240$ y **además** el correspondiente carácter cifrado también cumple
$33 \leq \text{cifrachar} \leq 240$. Si no se cumplen ambas condiciones en el fichero cifrado debe aparecer simplemente el carácter original del texto claro sin ninguna modificación. Y realizar un avance de registro.

16 Uso de dos operaciones de forma alternativa para cifrar. Sea (m_n) la sucesión que representa al mensaje claro y sea (c_n) la sucesión cifrante obtenida en el programa modelo proporcionado por el profesor.

Si c_n está en el rango $\{3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37\}$ se cifra el mensaje multiplicando módulo 256., es decir

$$e_n = m_n.c_n \qquad \text{Producto realizado en} \quad \mathbb{Z}/(256). \tag{3}$$

Si c_n **no** está en el rango indicado, se cifra tal y como se muestra en el programa modelo proporcionado por el profesor. Para sucesiones de bytes, pensado cada byte como 8 bits: Denotamos mediante $\oplus$ la suma en $\mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2 \times \mathbb{F}_2$ componente a componente. También se denomina suma bitwise. Realizamos

$$e_n = m_n \oplus c_n \qquad \text{suma bitwise.} \tag{4}$$

El mensaje cifrado es (e_n), que es el que debe escribirse en el fichero cifrado.

Nótese que la operación de descifrado ya no es igual a la de cifrado, y habrá que programarla aparte.
Para descifrar hay que multiplicar por los inversos de

$$\{3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27, 29, 31, 33, 35, 37\}$$

en $\mathbb{Z}/(256)$, que son respectivamente

$$\{171, 205, 183, 57, 163, 197, 239, 241, 27, 61, 167, 41, 19, 53, 223, 225, 139, 173\}.$$

17 Shriking generator. Generador automenguante. En vez de usar toda sucesión cifrante del programa modelo; se usará parte de ella y se cambia la operación de cifrado. Sea (c_n) la sucesión proporcionada por el programa modelo. Ahora operamos como sigue:

- Si $c_n \in \{0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22\}$, hacemos un avance en la sucesión (c_n), actualizamos el registro correspondiente, pero el valor c_n no se usa para nada.
- Si $c_n \in \{1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 21\}$, se cifra multiplicando módulo 256.

  $$e_n = m_n.c_n \qquad \text{Producto realizado en} \quad \mathbb{Z}/(256). \tag{5}$$

  En el descifrado habrá que multiplicar por los inversos de $\{1, 3, 5, 7, 9, 11, 13, 15, 17, 19, 20, 21\}$ en $\mathbb{Z}/(256)$, que son respectivamente $\{1, 171, 205, 183, 57, 163, 197, 239, 241, 27, 61\}$.
- Si $c_n \notin \{0, 1, 2, 3, \ldots, 21, 22, 23\}$, el valor c_n se usa para el cifrado y descifrado de la misma forma que en el programa modelo.

  $$e_n = m_n \oplus c_n \qquad \text{suma bitwise.} \tag{6}$$

18 Se usarán tres operaciones para cifrar. Sea (c_n) la sucesión proporcionada por el programa modelo. Ahora operamos como sigue:

- Si c_n es par, $c_n \in \{0, 2, 4, 6, 8, 10, 12, \ldots, 248, 250\}$, ciframos sumando módulo 256.

  $$e_n = m_n + c_n \qquad \text{Suma realizada en} \quad \mathbb{Z}/(256).$$

- Si $c_n \in \{5, 7, 9, 11, 13, 15, 17, 19, 21\}$, se cifra multiplicando módulo 256.

  $$e_n = m_n.c_n \qquad \text{Producto realizado en} \quad \mathbb{Z}/(256).$$

  En el descifrado habrá que multiplicar por los inversos de $\{5, 7, 9, 11, 13, 15, 17, 19, 20, 21\}$ en $\mathbb{Z}/(256)$, que son respectivamente $\{205, 183, 57, 163, 197, 239, 241, 27, 61\}$.
- Si c_n no está en ninguno de los conjuntos anteriores, c_n se usa para el cifrado y descifrado de la misma forma que en el programa modelo.

  $$e_n = m_n \oplus c_n \qquad \text{suma bitwise.}$$

El programa de descifrado es distinto del de cifrado.